PYTHON FOR DATA ENGINEERS

DATA VALIDATION & DATA QUALITY

PREPARED BY : KAVIRAJ T U

Why Data Quality Matters?

- In data engineering, the pipeline may run successfully, but the **data itself may be wrong**.

- Bad data leads to:
  ❌ Incorrect dashboards
  ❌ Wrong business decisions
  ❌ Broken machine learning models
  ❌ Loss of trust in the data platform

- The biggest risk is:
  ➜ Pipeline shows SUCCESS
  ➜ Data is silently corrupted

- Data quality is about **preventing bad data from entering the system**

What is Data Validation?

- **Data validation** is the process of checking whether incoming data:

  ➜ Matches expected structure
  ➜ Follows business rules
  ➜ Is complete and accurate

What is Data Quality?

- **Data quality** refers to how reliable and usable the data is.
- It is measured using key dimensions

- **<u>Core Data Quality Dimensions</u>**

Dimension	Meaning	Example
Completeness	No missing values	All orders have customer_id
Accuracy	Correct values	Amount is not negative
Consistency	Same across systems	Same user_id in CRM & billing
Validity	Matches format/rules	Email follows valid pattern
Uniqueness	No duplicates	order_id should be unique
Timeliness	Data arrives on time	Daily data before 2 AM

- **Common Real-World Data Issues**

  ❌ Null values in key columns
  ❌ Duplicate records
  ❌ Negative amounts
  ❌ Invalid dates
  ❌ Schema changes
  ❌ Unexpected data types

- **These issues break:**

  ➜ Aggregations
  ➜ Reports
  ➜ ML features
  ➜ Business logic

Types of Data Validation Checks

1. Schema Validation:

➔ Check column names and data types.
➔ Example:

- expected_columns = ["order_id", "customer_id", "amount"]

2. Null Checks:

- if record["customer_id"] is None:
  reject(record)

3. Range Checks

- if record["amount"] < 0:
  reject(record)

4. Format Checks

- import re
  if not re.match(r"[^@]+@[^@]+\.[^@]+", email):
  reject(record)

5. Uniqueness Checks

- if order_id in seen_orders:
  reject(record)

Real Pipeline Example

- **Scenario: Daily Orders Ingestion**
  → Incoming data:

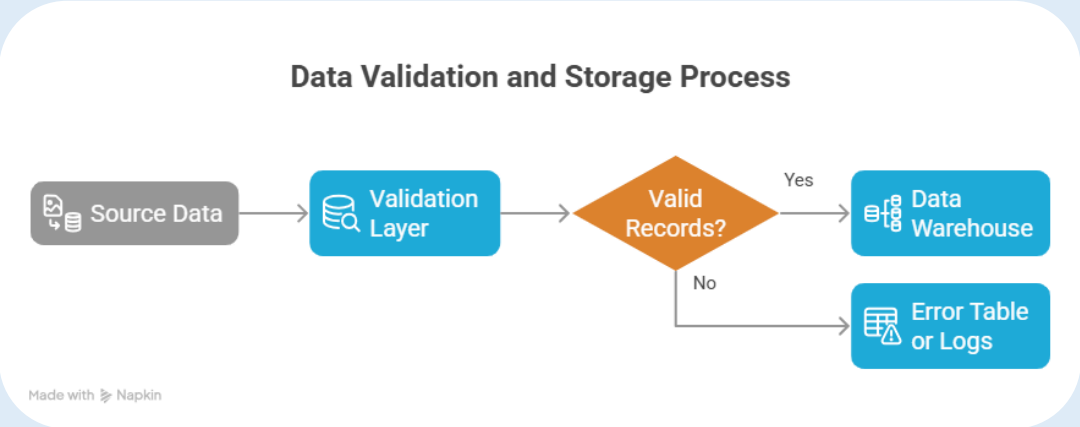
order_id	customer_id	amount
101	501	250
102	NULL	400
103	503	-50

- **Validation results:**

order_id	Issue
102	Missing customer_id
103	Negative amount

- **Action:**
  ➔ Valid records → Load
  ➔ Invalid records → Reject & log

Simple Validation Flow



Python Example: Basic Validation

QUERY

```python
def validate_record(record):
    if record["customer_id"] is None:
        return False
    if record["amount"] < 0:
        return False
    return True

valid_records = []
invalid_records = []

for record in data:
    if validate_record(record):
        valid_records.append(record)
    else:
        invalid_records.append(record)
```

What to Do with Bad Data

- Three common strategies:

Strategy	When to Use
Reject	Critical fields missing
Fix	Minor issues (trim spaces)
Flag	Load but mark as invalid

Data Quality Metrics

- **Track these daily:**

  ➜ Total records
  ➜ Valid records
  ➜ Invalid records
  ➜ Duplicate count
  ➜ Null percentage

- **Example:**

  ➜ Total: 100,000
  ➜ Valid: 98,500
  ➜ Invalid: 1,500
  ➜ Error rate: 1.5%

Production Best Practices

- Validate at ingestion
- Log all rejected records
- Track data quality metrics
- Set alert thresholds
- Store bad data separately
- Never silently drop records

Common Mistakes

- No validation layer
- Ignoring null values
- Hardcoded assumptions
- Silent data drops
- No quality metrics

Key Takeaways

- Data quality affects business decisions
- Pipelines must validate data early
- Track quality metrics continuously
- Never silently drop bad data
- Separate valid and invalid records

Interview Questions

1. What are the key data quality dimensions?

2. How do you handle invalid records?

3. Where should validation happen in a pipeline?

4. How do you detect duplicates?

5. What metrics indicate data quality issues?

 # Leave the answers in comment